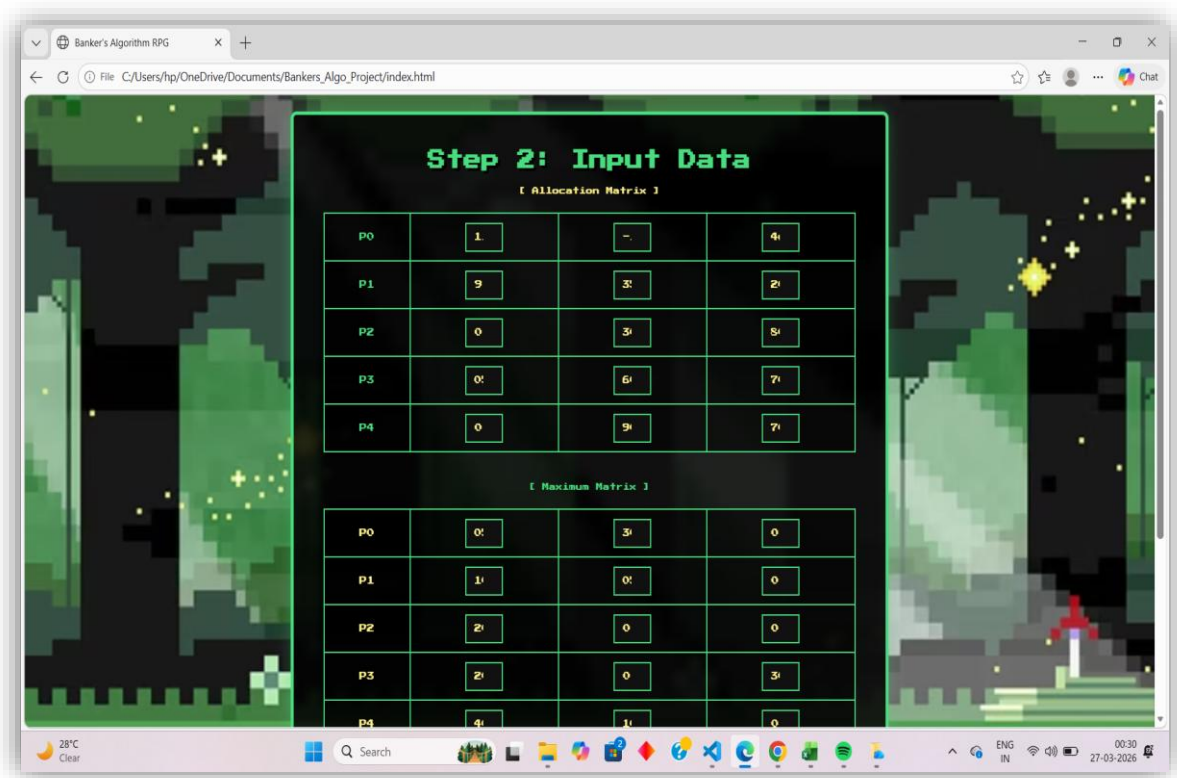
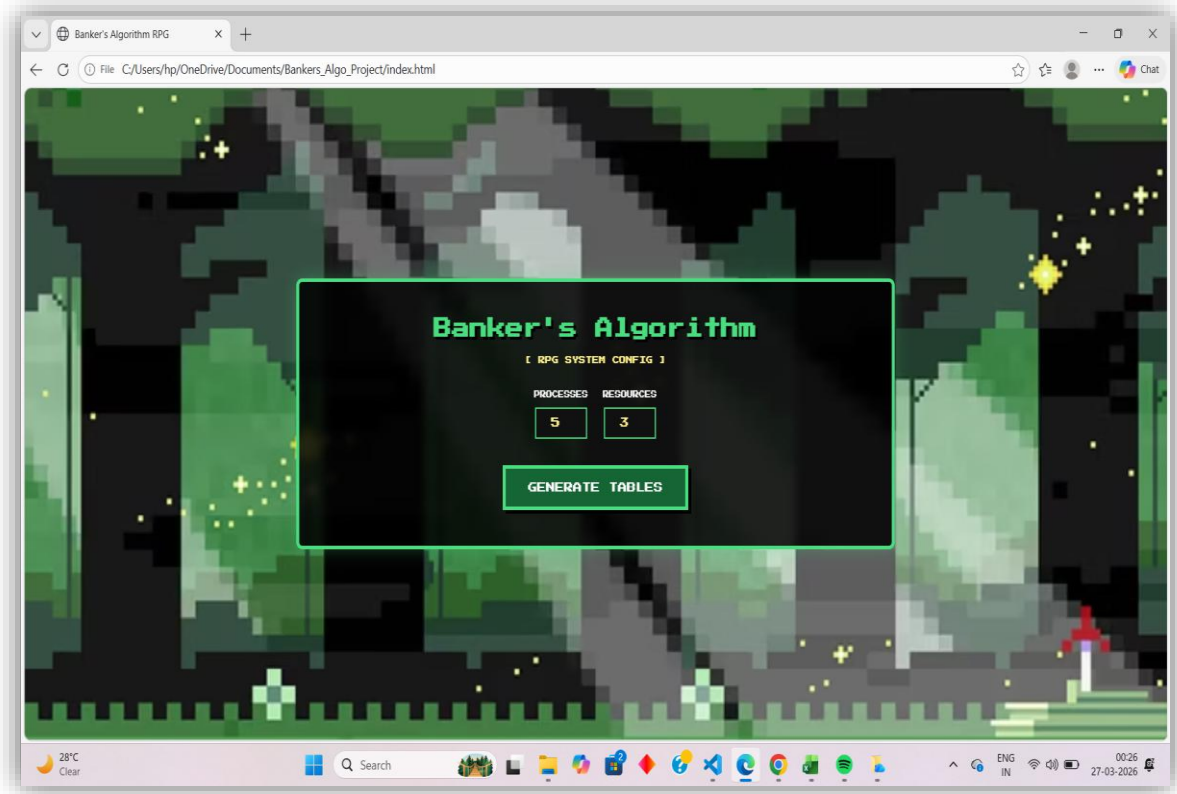
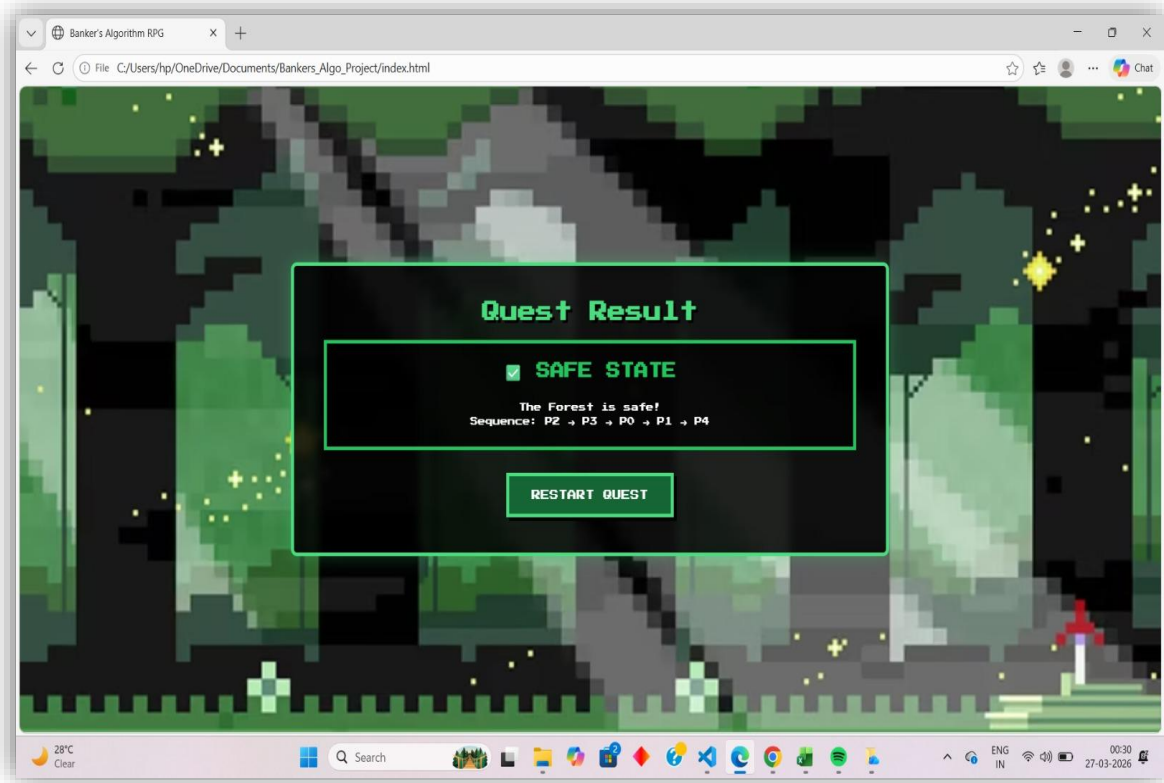
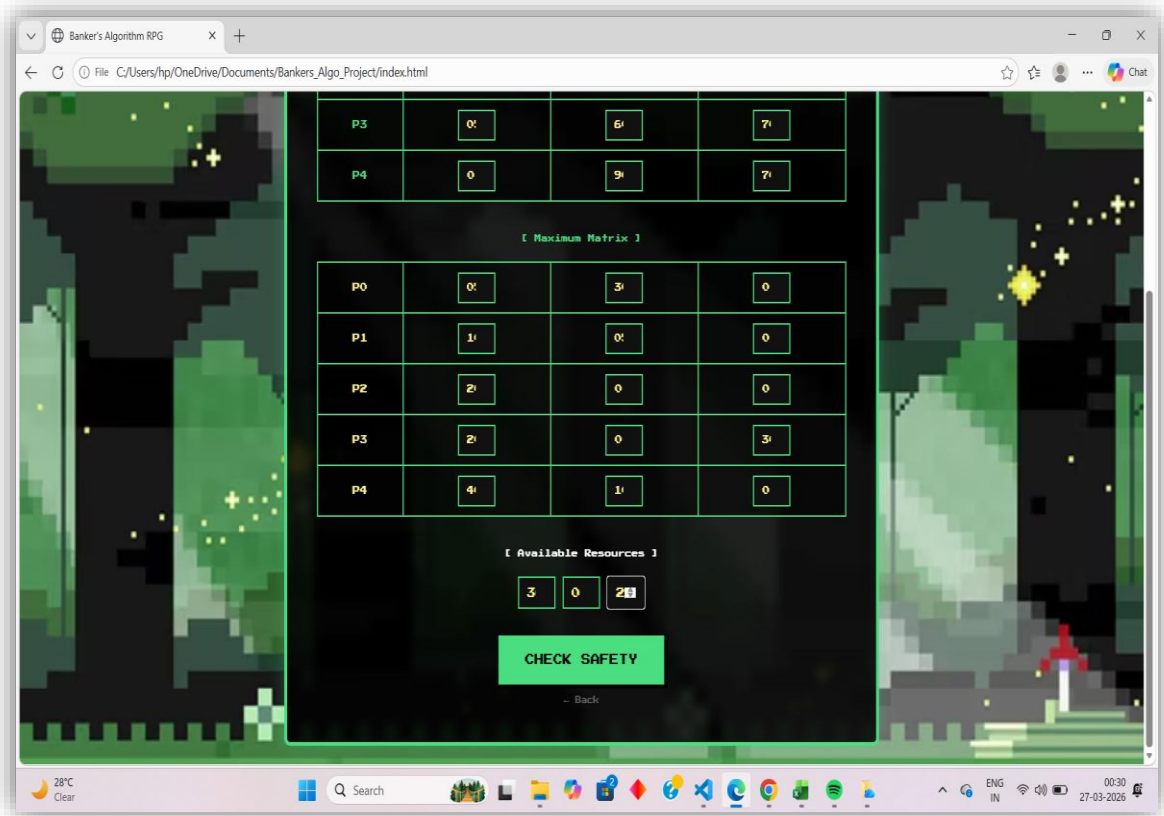
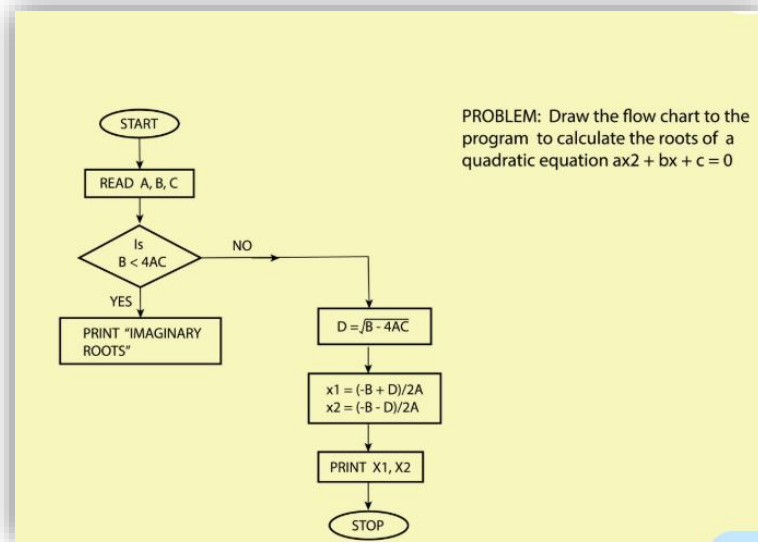


Design and implement a website on Bankers Algorithm Logic and Simulation.





The core logic of the Banker's Algorithm was first implemented in **C language** to ensure architectural accuracy. To make the process interactive and visual, a **Web-based Simulation** was also developed using HTML/CSS/JS.



CODE:

```
#include <stdio.h>

#include <stdbool.h>

int main() {

    int p, r, i, j, k;

    printf("--- BANKER'S ALGORITHM (SYSTEM LOGIC) ---\n");

    // Input: Number of Processes and Resources

    printf("Enter number of processes: ");

    scanf("%d", &p);

    printf("Enter number of resources: ");

    scanf("%d", &r);

    int alloc[p][r], max[p][r], avail[r], need[p][r];

    int safeSeq[p], work[r];

    bool finish[p];
```

```

// Input: Allocation Matrix
printf("\nEnter Allocation Matrix (Row by Row):\n");
for (i = 0; i < p; i++) {
    printf("P%d: ", i);
    for (j = 0; j < r; j++) {
        scanf("%d", &alloc[i][j]);
    }
}

// Input: Max Matrix and Calculate Need
printf("\nEnter Max Matrix (Row by Row):\n");
for (i = 0; i < p; i++) {
    printf("P%d: ", i);
    for (j = 0; j < r; j++) {
        scanf("%d", &max[i][j]);
        need[i][j] = max[i][j] - alloc[i][j];
    }
}

// Input: Available Resources
printf("\nEnter Available Resources: ");
for (j = 0; j < r; j++) {
    scanf("%d", &avail[j]);
    work[j] = avail[j];
}

// Initialize Finish array
for (i = 0; i < p; i++) {
    finish[i] = false;
}

```

```

}

// Safety Algorithm logic
int count = 0;
while (count < p) {
    bool found = false;
    for (i = 0; i < p; i++) {
        if (finish[i] == false) {
            // Check if Need <= Work for all resources
            for (j = 0; j < r; j++) {
                if (need[i][j] > work[j]) break;
            }
            // If all resources can be allocated
            if (j == r) {
                for (k = 0; k < r; k++) {
                    work[k] += alloc[i][k];
                }
                safeSeq[count++] = i;
                finish[i] = true;
                found = true;
            }
        }
    }
}

// If no process was found in a full cycle, system is unsafe
if (found == false) {
    printf("\nRESULT: System is NOT in a safe state (Deadlock Risk).\n");
}

```

```
        return 0;
    }
}

// Final Output: Safe Sequence
printf("\nRESULT: System is in a SAFE STATE.");
printf("\nSafe Sequence: ");
for (i = 0; i < p; i++) {
    printf("P%d", safeSeq[i]);
    if (i != p - 1) printf(" -> ");
}
printf("\n");
return 0;
}
```